

OmniParse AI Engine: Technical Architecture

Version 1.2.0 • Technical Documentation for RAG Verification

1. System Overview

OmniParse is a production-grade Document Ingestion and Retrieval-Augmented Generation (RAG) platform. It provides asynchronous, high-throughput parsing of structured and unstructured files (such as PDFs, Markdown, TXT, and CSV), transforming raw document layouts into clean, chunked vector representations. The engine is optimized for local-first operations but scales dynamically to cloud environments.

2. Ingestion Pipeline & Task Queue

The ingestion pipeline employs a decoupled architecture using FastAPI as the web layer and Celery (backed by Redis) as the asynchronous task worker. When a document is uploaded via the `/api/v1/documents/upload` endpoint, the API immediately returns a task ID and document ID, delegating the heavy parsing and layout analysis to the background workers. This prevents requests from timing out during the processing of complex multi-page documents.

Parameter	Configuration Value	Description
CELERY_BROKER_URL	redis://localhost:6379/0	Redis database 0 for task messages
CELERY_RESULT_BACKEND	redis://localhost:6379/0	Redis backend for task state persistence
Worker Concurrency	4 (Autoscaled)	Default concurrent tasks per node
Task Timeout	300 seconds	Hard timeout limit for parser workers

3. Vector Embeddings & Storage Details

Once text is extracted using **pdfplumber**, it is passed to the embedding pipeline. The default embedding model is **all-MiniLM-L6-v2** (384 dimensions) or optionally **Google Gemini Embeddings**. Vector representations and raw document chunks are indexed and stored in MongoDB.

Key storage characteristics:

- **Primary Database:** MongoDB is used to store document metadata, chunk text, and raw JSON extraction statistics.
- **Vector Indexes:** A cosine similarity index is built on the embedding field to enable high-speed semantic retrieval.
- **Chunk Strategy:** Text is parsed using recursive character splitting with a target chunk size of 500 characters and a 50-character overlap.

4. Model Performance Comparison Table

The following table compares the performance profiles of the supported embedding and query engines within the OmniParse architecture:

Embedding Model	Dimension	Latency (100 Chunks)	Retrieval Recall (Top-5)
all-MiniLM-L6-v2 (Local)	384	0.12 seconds	84.2%
bge-large-en-v1.5 (Local)	1024	0.45 seconds	89.7%
Google Gemini Text-004	768	0.28 seconds	92.5%

5. Query Execution Endpoint Example

A search query can be triggered by calling the following HTTP POST endpoint:

```
POST /api/v1/documents/{document_id}/query
Content-Type: application/json

{
  "query": "What database is used?",
  "top_k": 3
}
```